Auction schedule on the announce: session report

Worktree `auction-schedule`, branch `feat/auction-schedule`, 2026-10-25 to 2026-10-27. Plan: `doc/auction-schedule-plan.pdf` (approved 2026-10-25). The block "auction-schedule" of CLAUDE.md is implemented in full, verified by the module suites and by the manual checklist of `doc/frontend-edges.md` §7c, and committed on the branch (not merged).

1. What was built

Module	Change
announce-management (domain)	`GoodsDetails.auctionStartsAt/auctionEndsAt`; `AnnounceRules`: checkDetails (schedule only on AUCTION, start < end), resolveSchedule, checkSchedule (end <= expiry, end ahead), editedDetails (a published auction never loses its schedule); `AnnounceHandlers.publish/relist(clock, auctionDuration)` resolve the dates into `AnnouncePublished.details`, bean reads `announce.auction.default-duration` (7 d); `Announce.apply(AnnouncePublished)` takes the event's details. Codes: `auction-schedule-not-allowed`, `auction-schedule-order`, `auction-ends-after-expiry`, `auction-already-over`.
db-reactive	`announce_goods_detail.auction_starts_at / auction_ends_at` + key (`announce_table.sql`, bootstrap rebuilt); `AnnounceGoodsDetail` regenerated (restricted generator config), `upsert` re-added with the columns; `selectSummaries` and `AnnounceSummaryRow` expose both.
announce-management (read side)	Publish projection upserts the goods row next to the status; `AnnounceRows` maps both columns; `AnnounceSummary.KindFacts` gains them; `AnnounceDocument` emits `starts_epoch` / `ends_epoch` for local auctions (no indexer yml change).
announce-management-api	`GoodsDto` (+2), `AnnounceSummaryResponse.auctionEndsAt`, `AnnouncePublishedV1` and `AnnounceEditedV1` (+auctionStartsAt, auctionEndsAt, nullable); relay and mapper fill them.
order / order-api	`auctionAnnounceEvents` opens on the event's dates, falls back to publishedAt + default-duration capped by expiresAt; new `RescheduleAuction` -> `AuctionRescheduled` (OPEN, no standing bid, start < end; same window = no-op), apply, projection patch of start_time/end_time + audit row, `AuctionService.reschedule`, subscriber on `AnnounceEditedV1` (no auction yet / refused -> logged, consumed), `AuctionRescheduledV1` relayed to `auction.v1`.
gateway-api SPA	Goods form: two datetime-local pickers when AUCTION is chosen + help text; `validation.ts` mirrors the rules and drops the dates for other sale types; `AnnounceDetail` shows "Auction: starts .., ends .."; `AuctionPanel`: not started (info badge, disabled form, "Bidding opens .."), open ("started .. · ends in .."), closed ("ended .."), and the 404 state shows the announce's planned schedule plus the payout-account sentence; `AnnounceCard` shows "Ends {date}" for auctions of both sources; i18n en/da/sv; mock fixture `ann-auction-9`.
gateway-admin SPA	Auctions list gains a Starts column; the bids panel header shows "Runs start – end"; i18n en/da/sv.
Docs	`doc/order-context.md` §4 (opening and rescheduling rule, command row, public event), `doc/announce-management-context.md` §8 (fields, rule rows, read-model note), `doc/frontend-edges.md` §7c checklist executed and recorded.

2. Verification

Suite	Result
`mvn -pl db-reactive,announce-management-api,announce-management -P '!webapp' install`	80 of 81 pass; the one failure is pre-existing and unrelated: `ListingImportEndToEndTest.theRealV2Feeds..` (the real tradera feed of another worktree holds a duplicate item: seen 400, upserted 399).
`AnnounceTest`	5 new cases: explicit schedule kept; defaults (publish time + 7 d, start only, validity cap, configured duration); refusals (backwards, on FIXED_PRICE/GIVEAWAY, already over, after expiry); published edit carries the schedule and revalidates; relist keeps a window ahead and replaces one that is over.

Suite	Result
`AnnounceKindsEndToEndTest`	New test: scheduled AUCTION published -> response, read model (view and listing auctionEndsAt), index document (ends_epoch, starts_epoch) and AnnouncePublishedV1 carry the dates; defaulted AUCTION -> publishedAt + 7 d; 409 codes over HTTP.
`mvn -pl order -P '!webapp' test`	38 pass. AuctionTest: reschedule before a bid, no-op on the same window, refused after a bid / when closed / backwards, allowed again after a retraction. OrderModuleEndToEndTest: fallback opening (publishedAt, +7 d), scheduled opening on the event's dates, AnnounceEditedV1 reschedules before the first bid (AuctionRescheduledV1 on auction.v1), is refused after a bid (row keeps its end), both due auctions closed by the job.
`gateway-api npm test / i18n:check`	78 tests, 16 suites pass; i18n check OK. AuctionPanel spec: three time states and the 404 state with the planned dates; validation spec: end before start, end in the past, dates dropped for FIXED_PRICE.
`gateway-admin npm test / i18n:check`	65 tests, 17 suites pass; i18n check OK.
Manual check §7c	Executed 2026-10-27, every step as expected; section 3.

3. Manual check (frontend-edges §7c), 2026-10-27

Stack started from this worktree's jars (`scripts/start-stack.sh --no-build`) on the restored database, driven headlessly through the customer edge (8090) and the admin edge (8091) with curl; the SPA rendering of the panel states is covered by the jest specs and the served bundle carries the new keys.

Step	Seen
Rules behind the wizard	end before start: 409 `auction-schedule-order`; a schedule on FIXED_PRICE: 409 `auction-schedule-not-allowed`.
1. Post + publish	"Auction schedule check 7c" (`9608b514…`), start 2026-10-28 10:00, end 2026-11-04 10:00; publish 200.
2. Detail	`GET /api/announces/{id}`: PUBLISHED, goods.auctionStartsAt / auctionEndsAt as posted. Seller without payout account (first run): order log "Seller 1 has no enabled payout account", `GET /api/auctions/{id}` 404 = the panel's 404 state with the planned schedule. After `POST /api/sellers/me/account` (fake provider, VERIFIED) the next publish opened the auction: OPEN, startTime / endTime = the announce's dates.
3. Other member	bid 20000 as `exp21792849879`: 409 `auction-not-started` "The auction starts at 2026-10-28T10:00". Extra: `PUT /api/announces/{id}` with the start one day later -> 200, auction row startTime 2026-10-29 10:00 (RescheduleAuction from AnnounceEditedV1).
4. Cards	summary row carries auctionEndsAt 2026-11-04T10:00; `/api/search?q=schedule check&limit=2`: one hit, source local, sale_type AUCTION, starts_epoch 1793268000, ends_epoch 1793786400; `sale_type=AUCTION&source=local&sort=starts_epoch` sorts the local auctions by start.
5. Admin	`GET /api/admin/auctions`: 9608b514… OPEN, start 2026-10-29T10:00, end 2026-11-04T10:00.
Clean-up	announces of the first two runs withdrawn (auctions CANCELLED); 9608b514… left published.

Observed on the way, not part of this task: a publish within about a second of the seller's onboarding is still refused because the auction context's copy of `announce_seller_account` lags the onboarding response; the OCS searcher treats an unknown query parameter (`size`) as a field filter and answers zero hits (the SPA sends `limit`); `AnnounceExpiryJob` fails every 15 minutes with CommandSubjectDoesNotExistException for the seed row `2000018` and for `d3e5fad6…`.

4. Incident: the live database was wiped and restored

On 2026-10-25 at 22:15:21, while building a throwaway copy of the schema for an isolated stack, this session ran `mysql ... announce_reactive_db_as < db-reactive/sql/announce_reactive_db.sql`. That generated bootstrap begins with `DROP DATABASE IF EXISTS announce_reactive_db; CREATE DATABASE ...; USE announce_reactive_db;`, so the database named on the command line gave no protection: the live `announce_reactive_db` was dropped and recreated with empty tables while the

listing-availability session's stack was running on it. That session was informed at once; it confirmed that nothing of its own was written into the fresh schema and paused its work.

Recovery: the binary log is on (ROW, GTID off, 30-day retention) and complete; the database's creation is in `binlog.001303` (2026-09-19) and the DROP in `binlog.001327` at position 65962320. This session's permission classifier refused the replay on 2026-10-25; the user authorised it the next day and it ran on 2026-10-26 at about 19:50:

```
mysqlbinlog --read-from-remote-server -h127.0.0.1 -uroot --database=announce_reactive_db \
    --stop-position=65962320 binlog.001303 .. binlog.001327 | mysql -uroot --force announce_reactive_db
```

About 1.2 GB of row events replayed in eight minutes with zero SQL errors: 54 tables; announce 46, announce_goods_detail 32 (with this session's two columns), crawled_listing 1343 (962 ACTIVE / 200 STALE / 181 GONE, with the listing-availability migration), announce_user 24, announce_conversation 168, announce_chat_message 13999, 17 projection checkpoints. The listing-availability session verified its tables. The empty throwaway database `announce_reactive_db_as` was dropped. Lesson kept in memory: never feed `announce_reactive_db.sql` to the server that holds the live database; use a MySQL container for a throwaway schema.

Side effect of the check stack: it started before the other session had moved its 2026-10-26 crawl feeds out of the import directory, imported `dba_202610261913.jsonl` (445 DBA rows, indexed without the new index bookkeeping) and rejected two status feeds into `failed/`; the other session was told exactly what happened.

5. State of the branch

- One commit on `feat/auction-schedule` with the implementation, the docs, the plan PDF and this report; not merged into `announce-jo-branch`.
- Out of scope and untouched: reserve / buy-now prices, auction templates, time zones beyond the server's local time, notifications.
- Follow-ups worth a block of their own: the onboarding lag before OpenAuction (retry or ordering), and the expiry job on seed rows without events.